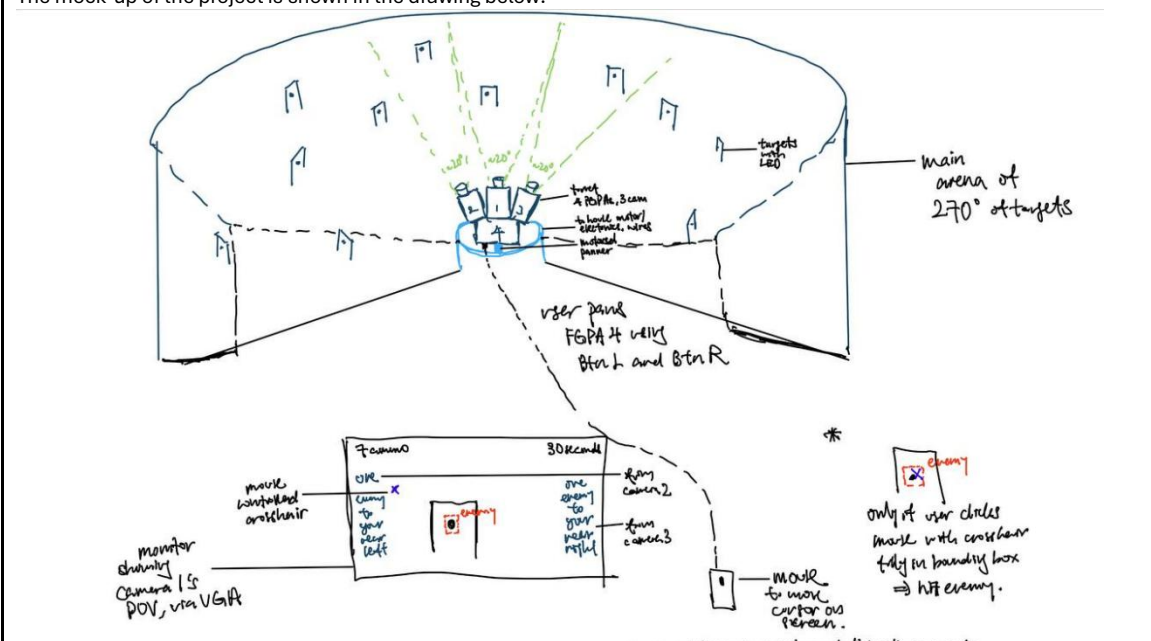


I-FDP PROPOSAL DOCUMENT

Group ID: S1_02	Description (Pictures / Drawings / Sketches / Gantt Charts are optional and not necessary)
Group Theme: FPGA-Based Interactive Turret Game Using Computer Vision	This project is an interactive turret system that detects, tracks, and “shoots” LED-based targets within a 270° field of view using computer vision and user input. The system integrates four Basys3 FPGAs and three OV7670 cameras, operating under one bitstream, to perform object detection, game logic, and user interaction in real time. Goal: shoot as many targets as possible before time and ammo run out. The mock-up of the project is shown in the drawing below:  Setup: The turret is built using 4 FPGAs and 3 OV7670 cameras, placed at the centre of the area. - Each camera runs an object detection algorithm using computer vision, which detects targets within its field of view. - 3 FPGAs will be dedicated to one camera each. - o 7 segment display of each FPGA will show number of targets detected - o In addition, the main camera (centre FPGA1) will provide live feed via VGA to monitor display - The 4th FPGA acting as the game master and main control unit: - o Buttons to control the panning of the turret along the horizontal axis - o LEDs to indicate ammo left (max of 15, decreasing by one every left-click of mouse) - o 7 segment display to show number of targets hit - o Speaker to provide live feedback on the status of the game (e.g. “HIT”) - o LED controller drives the targets The arena is a 270-degrees spanning field, with “targets” scattered across. These targets will each have an LED placed on it, for the camera to detect autonomously. The monitor will be the main display for the player: - Displays the live feed of the centre camera’s POV, which is on FPGA 1 - The bounding boxes for targets in view done by the camera’s object detection algorithm - Mouse cursor of the player, controlled by a mouse - Early-warning system: Indicate presence of enemies that are outside of view, detected by left camera (on FPGA 2) and right camera (on FPGA 3) - Remaining time of the current round <u>User-Mode (Player manually aims via mouse and turret controls):</u> At each round, a certain number of targets will light up. Player pans the turret to find all the targets. To hit the target, the user simply moves the cursor on the monitor display to be within the target’s bounding box and then left-click. There is limited amount of time to hit the targets for each round. There is also a limited number of ammos: once the ammo count reaches zero or timer reaches zero, the round ends and the current score will be added to the final score. After 3 rounds, a final counter of targets successfully hit will be displayed on the 4th FPGA’s 7 segment display. <u>Auto-Mode (System autonomously aims and fires using camera detections):</u> This is the CPU “auto assist” mode where the game is completed autonomously. This will be the target to score for single players. Upon the detection of a target by FPGA 1, using Verilog logic by reading the current cursor coordinates and the bounding box’s centroid coordinates,

	the cursor will move autonomously, and inputs a click upon entering the bounding box, hitting the enemy. This mode can be set from easy to hard for the players, where difficulty is determined by the amount of time needed to hit the target by the CPU. <u>Mode switching:</u> Difficulties and modes can be switched on the 4th FGPA, using the onboard switches. (i.e. user/auto mode, and higher difficulties have lesser ammo and lesser time)
Group Member 1: JOEL KU	Member Responsibility: FPGA4 tasks (LED target control, speaker, 7-segment, buttons/switches, turret control, inter-board and other communications) Week 8 tasks: Design FSM for LED control, design communication structure, logic and method, plan inter-FPGA and LED control wiring, physically verify max LED brightness based on FPGA GPIO current limits, design external circuit for LED power if required Week 9 tasks: Implement LED control FSM and communication module, test both with FPGAs and LEDs, integrate other internal FPGA components (7-segment, buttons, switches), build LED boards, create PWM module for servo control Week 10 tasks: Implement error handling for communication, randomize LED outputs based on game difficulty, ensure the correct information is being passed to FPGA1, ensure switches set the correct mode for each FPGA, integrate PWM module for turret control Week 11 tasks: Implement speaker control, ensure turret movement is smooth, finalize LED control strategy based on game logic
Group Member 2: JOSHUA YEO WEE TZE	Member Responsibility: FPGA1 tasks - User input integration (mouse overlay and cursor), overlap logic and user-mode FSM. Week 8 tasks: Implement mouse input for FPGA and current cursor position tracking, render crosshair to show current cursor position on the VGA output. Week 9 tasks: Implement debouncing (and anti-holding) functions for all inputs (mouse, buttons, switches), overlap checks for crosshair and bounding box, integrate with inter-FPGA communications for game state inputs (buttons to move turret and mouse to move cursor) Week 10 tasks: integrate with camera to get live bounding box output for mouse integration testing, with turret/servo control for full user control. integrate with game state to ensure correct inputs Week 11 tasks: Integrate with game state logic to update state after every user input, finalize final user operation (buttons and mouse), ensuring user-friendliness of user operation
Group Member 3: KENNETH WONG CUN WI	Member Responsibility: Object detection algorithm, target hit logic and auto-mode aiming logic. Week 8 tasks: Research on edge detection logic using time and memory efficient algorithm for quick edge detection. Week 9 tasks: Complete object detection algorithm and integrate the bounding box output into one display output. Implement a hitbox detection logic given a cursor's position. Week 10 tasks: Implement the auto-mode logic and integrate edge detection into early-warning system. Week 11 tasks: Add difficulty to game state FSM, integrate with others to finalize final VGA output (includes mouse, stats, bounding boxes and camera output)
Group Member 4: TAN KAI CONG	Member Responsibility: VGA camera pipeline, early warning system (FPGA 1 to 3), game state management, inter-FPGA synchronisation Week 8 tasks: Setup VGA output. Research camera models and communication / image capture. Purchase camera, wire up camera connections and test image capturing process onto a monitor via VGA. Week 9 tasks: Modularize image capture, decide on suitable colour depth and image output resolution to optimise memory space. Design the early warning system. Design FSM for game states, with ammo and time tracking, ensure FSM stability Week 10 tasks: Calibrate cameras (focus, brightness); integrate FSM with user inputs; implement inter-FPGA communication to ensure consistent game state updates and output displays (7-segment, LEDs). Week 11 tasks: Ensure camera outputs are well integrated with other display tasks (mouse aiming, game stats, early-warning)
Group Integration:	Week 9 integration: Perform unit tests for all modules. Integrate object detection, camera input, and VGA output for feedback and display. Integrate mouse input and cursor overlay with overlap detection. Establish communication bus between FPGAs. Test all I/Os (speaker, LEDs, switches). Week 10 integration: Conduct partial system integration by connecting camera outputs, VGA, and game logic pipeline. Test both user and auto modes with a single target. Integrate inter-FPGA communication (1–3 and to 4). Test LED configurations for targets. Week 11 integration: Integrate all FPGAs and peripherals into one coherent and visually polished product. Demo and stress test.